

Casino Game Suite

Project Documentation

Course	CS-A1123 Basics in Programming Y2
Student	Ang Yong Yi, Jatlyson
Student Number	104083758
Date	7 May 2026

Aalto University

School of Computer Science

Contents

1	General Description	3
2	User Instructions	3
2.1	Launching the Program	3
2.2	Main Lobby	3
2.3	Player Setup (Deck Casino)	3
2.4	Playing Deck Casino	4
2.5	Playing Blackjack	4
2.6	Playing Baccarat	4
3	External Libraries	4
4	Structure of the Program	5
4.1	Architectural Pattern	5
4.2	Model Layer Classes	5
4.3	View Layer Classes	6
4.4	Controller	7
4.5	Class Relations	7
5	Algorithms	7
5.1	Take Validation: Subset-Sum with Multi-Group Backtracking	7
5.2	AI Opponent: Six-Priority Decision Tree (Hard Difficulty)	8
5.3	Blackjack Dealer: Basic Strategy Table	9
5.4	Baccarat Third-Card Rules	9
5.5	Score Calculation	9
6	Data Structures	10
6.1	Lists	10
6.2	Frozensets	10
6.3	Dictionaries	10
6.4	Mutable vs Immutable	10
7	Files	10
7.1	Save File (save_{names}.json)	10
7.2	Bankroll File (bankrolls.json)	11
7.3	No Configuration Files	11
8	Testing	12
8.1	Test Suite Overview	12
8.2	Testing Methodology	12
8.3	Known Test Coverage Gaps	12
9	Known Shortcomings and Flaws	13
10	Three Best and Three Weakest Parts	13
10.1	Three Strongest Parts	13
10.2	Three Weakest Parts	14

11 Changes to the Original Plan	14
11.1 Planned Features Delivered	14
11.2 Implementation Differences from the Plan	15
11.3 Additional Features Beyond the Plan	15
12 Realised Order and Schedule	15
13 Assessment of the Final Result	16
14 References	17

1 General Description

This project is a multi-game casino suite implemented in Python with a graphical user interface built using PyQt6. The suite contains three distinct card games:

- **Deck Casino** (Finnish Kasino): a card-taking game where players capture table cards by playing a hand card whose value matches a subset sum of cards on the table. This is the primary game, featuring an AI opponent with three difficulty levels, full save/load support, and a move-history panel with improvement tips.
- **Blackjack**: standard casino Blackjack against an AI dealer. Supports Hit, Stand, Double Down, and Split.
- **Baccarat (Punto Banco)**: a fully deterministic game where all decisions are resolved by fixed third-card rules after the player places a bet.

Compared to the original project plan, the scope expanded in several areas: a bankroll system shared across Blackjack and Baccarat was added, an in-game tutorial overlay was implemented, card animations (deal and flip) were introduced, and a move-history panel with automated advice tips was completed. The core Deck Casino rules, scoring, and AI opponent described in the plan were implemented as specified.

This project accomplishes the hard level for the initial intended difficulty level for Deck Casino. The take-validation algorithm supports multi-group takes (a single played card capturing several disjoint groups simultaneously), the AI implements a six-priority strategic decision tree with sweep-avoidance placement logic, the GUI rebuilds from game state on every action with animated transitions, and the test suite contains 242 unit tests covering all model classes.

2 User Instructions

2.1 Launching the Program

1. Install the only external dependency: `pip install PyQt6`
2. Run the application from the project root: `python main.py`

Python 3.10 or higher is required. No additional setup, configuration files, or data files are needed.

2.2 Main Lobby

After launch, the lobby screen appears with three game buttons: *Deck Casino*, *Blackjack*, and *Baccarat*. Clicking any button opens a player-setup dialog before the game begins.

2.3 Player Setup (Deck Casino)

- **Player vs Player**: choose 2, 3, or 4 players and enter names. All are human-controlled.
- **Player vs Computer**: enter your name and select AI difficulty (Easy, Medium, or Hard).

If a save file exists for the chosen player roster, a prompt offers to resume the saved game.

2.4 Playing Deck Casino

Each turn the current player selects a card from their hand (left panel) and either:

- **Take:** click one or more table cards whose values sum to the hand card's value, then click *Take*. Multiple disjoint groups may be selected at once.
- **Place:** click *Place* to put the hand card onto the table without capturing anything.

Additional controls:

- **Hint Mode** (toggle): shows the numerical hand/table value badge on each card.
- **Tutorial:** step-by-step overlay explaining the rules.
- **Save:** writes the current game state to a JSON file keyed by player roster.
- **History:** opens the move-history panel showing every action with an improvement tip.

AI turns execute automatically after the human's turn. A round ends when the stock pile and all hands are empty; scores are tallied and a new round begins. The first player to reach 16 points wins.

2.5 Playing Blackjack

1. Enter a bet amount and click *Deal*.
2. Choose *Hit*, *Stand*, *Double Down*, or *Split* (when eligible).
3. The dealer plays automatically once the player stands or busts.
4. Winnings are credited to the bankroll and persisted for future sessions.

2.6 Playing Baccarat

1. Choose a bet side: *Punto*, *Banco*, or *Tie*.
2. Enter a bet amount and click *Deal*.
3. All third-card drawing happens automatically; the result is displayed immediately.

3 External Libraries

The only external library used is **PyQt6** (version 6.x). It provides the entire graphical user interface: windows, widgets, layouts, signals/slots, and the **QPainter** API used to draw card art. No image files are included; all visual elements are rendered programmatically.

The Python standard library is used for **json** (save/load), **os** (file paths), **random** (deck shuffling and AI randomness in easy mode), **re** (filename sanitisation), and **itertools** (combination generation in the take algorithm).

No other third-party libraries are used. The test suite is run with the Python standard library `unittest` module, which requires no additional installation.

4 Structure of the Program

4.1 Architectural Pattern

The program follows a strict Model–View–Controller (MVC) pattern with three layers:

- **Model** (`model/`): pure game logic with no PyQt6 imports. All rules, scoring, AI strategy, bet validation, and file I/O live here.
- **View** (`view/`): PyQt6 widgets. Views contain zero game logic; they rebuild entirely from game state on each action.
- **Controller** (`controller/`): a thin `GameManager` that holds the active game instance and player roster.

Signal flow: views emit Qt signals → `MainWindow` catches them → calls model methods → calls `view.refresh(game)`. This makes the model fully testable without any GUI.

4.2 Model Layer Classes

Card	Immutable value object for a single playing card. Implements the dual-value design central to Deck Casino: <code>hand_value()</code> returns the value when the card is played from hand (inflated for special cards), while <code>table_value()</code> returns the value when the card sits on the table. Implements <code>__hash__</code> and <code>__eq__</code> so cards can safely be stored in sets and used as frozenset members.
Deck	A 52-card ordered collection. Dealing pops from the end of the list (the “top” of the deck). Raises <code>ValueError</code> if asked to deal more cards than remain.
Hand	A mutable card aggregator for one participant. Provides <code>get_value()</code> (sum of <code>hand_value()</code> for all cards, used in Deck Casino) and <code>blackjack_value()</code> (Ace-flexible total, used in Blackjack). Both return copies of the internal card list to prevent external mutation.
Player	Represents a human or AI participant. A single <code>Player</code> object is used across all three games, carrying the hand, collection, sweep count, cumulative score, and bankroll. <code>reset_for_round()</code> clears per-round state while preserving the cumulative score.
CasinoTakeAlgorithm	Stateless class (only <code>@staticmethod</code> methods) containing the take-validation logic. <code>get_valid_takes()</code> enumerates all subsets of table cards that sum to the played card’s hand value. <code>is_valid_take()</code> checks whether the player’s chosen set can be partitioned into one or more non-overlapping valid subsets via the recursive <code>_can_cover()</code> helper.
DeckCasinoGame	Main Deck Casino game loop. Manages the turn order, table, stock pile,

and last taker. `play_card(card, take)` is the single entry point for a turn. `calculate_scores()` awards all six point categories at the end of a round.

BlackjackGame

One round of Blackjack. Tracks game phase ("betting" → "player" → "dealer" → "done"). Uses parallel lists (`_hands` and `_bets`) to support split hands.

AIDealer Encapsulates Blackjack dealer rules (hit below 17, stand at 17+) and a hard-total basic-strategy table.

BaccaratGame

One round of Punto Banco. The round is fully deterministic once `start_round()` is called; `draw_third_card()` applies the standard rule table.

AIOpponent Stateless class with three difficulty strategies for Deck Casino. All methods are `@staticmethod`; no instance is ever created.

BettingSystem

Stateless class for bet validation and payout calculation for both Blackjack and Baccarat.

FileManager

Handles JSON serialisation and deserialisation of full `DeckCasinoGame` state, bankroll persistence, and move-history recording. Also provides `find_case_insensitive`, which detects when a newly entered player name differs only in capitalisation from an existing bankroll entry (e.g. "alice" vs. "Alice") and returns the stored key so `MainWindow` can prompt the player to load that saved balance.

4.3 View Layer Classes

MainWindow The application shell. Holds a `QStackedWidget` that pages between the lobby, Deck Casino, Blackjack, Baccarat, and history views. Owns all primary signal-to-model wiring.

LobbyView The game-selection home screen.

DeckCasinoView

The Deck Casino table. Maintains selection state (one hand card, zero or more table cards) and emits `take_requested` and `place_requested` signals.

BlackjackView

The Blackjack interface. Drives the game directly (betting, dealing, and action buttons) without back-signals to `MainWindow`.

BaccaratView

The Baccarat bet and result screen.

CardWidget A `QWidget` subclass that draws a single playing card using `QPainter`. Provides `animate_deal()` and `animate_flip()` via `QTimer`.

drawn_assets

Shared QPainter helpers: `SweepFlashOverlay`, `RoundResultOverlay`, `PointToastOverlay`, `AvatarWidget`, and the application icon.

GameHistoryView

Displays the recorded move history for the current Deck Casino session, with the auto-generated improvement tip for each move.

DeckCasinoTutorial

Step-by-step overlay guide for new players, shown over the Deck Casino table.

PlayerBadgeWidget

Compact widget showing a player's name, score, and sweep count in the Deck Casino sidebar.

4.4 Controller

`GameManager` is intentionally thin: it holds the active game instance and the player roster so that `MainWindow` can request a new Deck Casino game without needing to know construction details. Blackjack and Baccarat bypass `GameManager` and are launched directly by `MainWindow`, because they are single-player and single-round and do not require the roster management that Deck Casino needs.

4.5 Class Relations

- `DeckCasinoGame` *owns* a `Deck` and a list of `Player` objects.
- Each `Player` *owns* a `Hand`.
- `DeckCasinoGame` *uses* `CasinoTakeAlgorithm` (static calls, no ownership).
- `MainWindow` *owns* `GameManager` and all view objects.
- `BlackjackGame` and `BaccaratGame` each *own* a `Deck`, one or more `Hand` objects, and a `Player` reference. They *use* `BettingSystem` statically.
- `AIOpponent` and `BettingSystem` have no ownership relations; they are stateless utility classes called by game objects.

5 Algorithms

5.1 Take Validation: Subset-Sum with Multi-Group Backtracking

The most algorithmically interesting part of the program is determining whether a player's chosen set of table cards constitutes a valid take in Deck Casino.

The basic rule is: a played card with hand value T may take any non-empty subset of table cards whose table values sum to T . The challenge is the *multi-group* extension: a single played card may simultaneously capture several disjoint groups, each summing to

T independently. For example, playing a Jack (value 11) against a table of $\{6, 5, J, A, 10\}$ allows taking all five cards at once because $6 + 5 = 11$, $J = 11$, and $10 + A = 10 + 1 = 11$.

Phase 1: Enumerate valid subsets. `get_valid_takes(played_card, table_cards)` iterates over all non-empty subsets of the table (using `itertools.combinations`) and collects every subset whose sum of `table_value()` equals the played card's `hand_value()`.

Phase 2: Check partition coverage. `is_valid_take(played_card, chosen_cards, table_cards)` must decide whether the player's chosen set C can be partitioned into valid subsets. This is done by `_can_cover(remaining, valid_subsets)`, a recursive backtracker:

```
def _can_cover(remaining: frozenset, valid_subsets: list[frozenset]) -> bool:
    if not remaining:
        return True                                # base case: all covered
    for subset in valid_subsets:
        if subset.issubset(remaining):
            if CasinoTakeAlgorithm._can_cover(remaining - subset, valid_subsets):
                return True
    return False
```

The function tries each valid subset that fits within the remaining uncovered cards, recurses on what is left, and backtracks if that branch fails. The recursion terminates because each successful branch strictly reduces `remaining`.

An alternative would have been a greedy approach (always picking the largest matching subset). However, greedy fails on adversarial inputs where a large subset “blocks” a valid complete partition. The backtracking approach is correct for all inputs and performs well in practice because the table rarely exceeds eight cards and the valid-subset count is small.

5.2 AI Opponent: Six-Priority Decision Tree (Hard Difficulty)

The hard AI evaluates all legal (`card`, `take`) pairs and selects the best action according to a strict priority ordering:

1. **Sweep:** if any card can take all table cards at once, take the sweep using the lowest-value hand card (preserving high-value cards for later).
2. **Prize capture:** if any take includes the Diamond-10 or Spade-2, prefer the one that takes the most cards alongside the prize card.
3. **Maximise cards taken:** among remaining options, choose the take that collects the most cards.
4. **Special-card preference:** among equally-sized takes, prefer one that contains a special card (Ace, Diamond-10, or Spade-2).
5. **Sweep-safe placement:** if no take is available, choose the card to place that does not create a sweepable table state for the opponent.

6. **Lowest hand value:** if no safe placement exists, place the card with the smallest hand value to minimise loss.

Priority 5 is implemented by checking, for every candidate placement card, whether the resulting table ($\text{current table} \cup \{\text{card}\}$) can be swept by any hand value in the range $[2, 16]$. This check calls the same `_can_cover` backtracker as the take validation, making it consistent with the game rules.

The medium AI uses only priorities 1, 3, and 6. The easy AI picks a random legal move.

An alternative AI design could have used minimax or Monte Carlo tree search. These were rejected because the game state space is large (combinatorial card arrangements) and partial information (opponent's hand is hidden) makes minimax difficult to apply correctly. The priority-tree approach encodes expert knowledge about the game and runs in milliseconds even for large tables.

5.3 Blackjack Dealer: Basic Strategy Table

The `AIDealer` builds a hard-total basic-strategy table at first use, mapping `(player_total, dealer_upcard)` pairs to recommended actions (*hit*, *stand*, or *double*). The dealer itself always hits below 17 and stands at 17 or higher. The table follows the standard casino basic-strategy derivation, which minimises the house edge over many hands.

5.4 Baccarat Third-Card Rules

Baccarat is fully deterministic. After dealing two cards to each side:

- If either side totals 8 or 9 (a *natural*), the round ends immediately.
- Punto draws a third card if its total is ≤ 5 ; stands on 6 or 7.
- Banco's drawing decision depends on its current total and, if Punto drew, the value of Punto's third card, according to the standard rule table encoded in `_banco_draws()`.

No player decision is required after the bet is placed.

5.5 Score Calculation

At the end of each Deck Casino round, six point categories are awarded:

Category	Points	Tie rule
Each sweep	+1 per sweep	N/A
Each Ace collected	+1 per Ace	N/A
Most cards	+1	No award on tie
Most Spades	+2	No award on tie
Diamond-10 holder	+2	First collector wins
Spade-2 holder	+1	First collector wins

The game ends when any player reaches 16 points total across all rounds.

6 Data Structures

6.1 Lists

Python `list` is the primary sequence type throughout the program. The deck's draw pile is a list; dealing pops from the end (index -1), which is $O(1)$ and avoids the $O(n)$ cost of popping from index 0. Table cards, player collections, hand cards, and move history are all lists. A list was chosen over a deque because random access and slicing (needed for collection scoring) are $O(1)$ on a list.

6.2 Frozensets

Take groups are represented as `frozenset[Card]` objects. Frozensets are used because:

- Ordering is irrelevant for a group of taken cards.
- Frozensets are hashable, enabling them to be stored in lists of valid takes and used as dictionary keys if needed.
- Set membership testing (`issubset`) is $O(|S|)$ and is called many times during back-tracking.

Cards themselves implement `__hash__` based on (`suit`, `rank`), making them safe frozenset members.

6.3 Dictionaries

`FileManager` uses a plain Python `dict` for the JSON save format and for `bankrolls.json` (a mapping from player name to bankroll amount). The AI basic-strategy table is stored as a `dict[tuple[int, int], str]`, keyed on (`player_total`, `dealer_upcard`), which provides $O(1)$ lookup per turn.

6.4 Mutable vs Immutable

`Card` is immutable (private attributes, no setters). `Hand` and `Player` are mutable. All property accessors that return internal collections return copies (`list(self._cards)`) so callers cannot accidentally mutate game state. This was a deliberate design decision: the view layer must never mutate the model directly.

7 Files

7.1 Save File (`save_{names}.json`)

When the player saves a Deck Casino game, a JSON file is written to the project root. The filename encodes the player roster (e.g. `save_Alice-Bob.json`) so different groups can maintain separate saves. The file format is:

```
{
  "turn_index": 0,
  "table_cards": [ {"suit": "Hearts", "rank": "7"}, ... ],
  "stock":       [ {"suit": "Clubs",   "rank": "3"}, ... ],
```

```
"players": [
  {
    "name": "Alice",
    "is_ai": false,
    "difficulty": "hard",
    "total_score": 5,
    "sweeps": 1,
    "bankroll": 1000,
    "hand": [ {"suit": "Spades", "rank": "K"}, ... ],
    "collection": [ {"suit": "Hearts", "rank": "A"}, ... ]
  },
  ...
],
"move_history": [
  {
    "player": "Alice",
    "played": "7 of Hearts",
    "action": "take",
    "took": ["7 of Clubs"],
    "advice": "Good move -- you captured high-value card(s):
      ..."
  },
  ...
]
```

Cards are serialised as `{"suit": ..., "rank": ...}` objects. The stock pile is stored in deal order (last element = top of deck) so the game can be resumed with identical card ordering.

7.2 Bankroll File (bankrolls.json)

A separate JSON file stores per-player bankrolls that persist across Blackjack and Baccarat sessions:

```
{
  "Alice": 1350,
  "Bob": 800
}
```

The file is updated after every Blackjack or Baccarat round. Different player names have independent entries.

7.3 No Configuration Files

The program requires no user-created configuration files. It can be run immediately after `pip install PyQt6`.

8 Testing

8.1 Test Suite Overview

The project contains 242 unit tests across five test files, all runnable with:

```
python -m unittest discover tests/
```

File	Scope	Tests
test_deck_casino.py	Card, Deck, Hand, Player	88
test_algorithm_and_game.py	CasinoTakeAlgorithm, DeckCasinoGame	72
test_week4.py	Integration: take + game state	26
test_week5.py	BettingSystem, BlackjackGame, BaccaratGame, AIDealer	43
test_week7_ai.py	AIOpponent (unit + property + integration)	13

All 242 tests pass as of the submission date.

8.2 Testing Methodology

Unit tests cover each model class in isolation. Critical boundary conditions are tested explicitly: dealing from an empty deck, removing a card not in hand, betting more than the bankroll, calling `draw_third_card()` after a natural, and so on.

Integration tests in `test_algorithm_and_game.py` inject controlled card states directly into the game (bypassing the random deck) using the `inject_state()` helper, then verify that the full turn pipeline (`play_card` → take validation → collection update → score check) produces the correct outcome.

Property tests in `test_week7_ai.py` run 10 complete AI vs AI games and assert that the hard AI never leaves a sweepable table state when a safe placement option existed. This checks a strategic invariant that is difficult to specify with a single unit test.

AI validity tests verify that at every difficulty level, the AI's chosen move is accepted by `play_card()` without raising `ValueError` for 100 consecutive moves.

Testing during development was done incrementally: each new class was covered before the next was built. The `inject_state()` pattern allowed complex scenarios (multi-group takes, sweep detection) to be tested deterministically without relying on a shuffled deck.

8.3 Known Test Coverage Gaps

The view layer (PyQt6 widgets) is not unit tested. GUI behaviour was verified manually by running the application. The `FileManager` is tested indirectly through game integration but does not have dedicated save/load round-trip tests. The Baccarat third-card rule table was tested through game-level `get_result()` assertions rather than the internal `_banco_draws()` method directly.

9 Known Shortcomings and Flaws

1. **Baccarat view is minimal.** The Baccarat interface lacks deal animations and detailed card display compared to Deck Casino and Blackjack. The model and rules are correct, but the visual presentation is less polished.
2. **No network or hot-seat turn management.** In Player vs Player mode, all players share the same screen. There is no hidden-hand mechanism, so players must agree to look away during each other's turns. A proper hot-seat mode would display a "pass the device" screen between turns.
3. **AI difficulty affects Deck Casino only.** The Blackjack AI dealer always uses the same rule (hit below 17); there is no easy/hard mode for Blackjack. This was a deliberate scope decision but limits variety.
4. **The take algorithm does not prune.** `get_valid_takes()` enumerates all $2^n - 1$ non-empty subsets of the table (where n is the number of table cards). For a full table of eight cards this is 255 subsets, which is fast, but for hypothetical larger tables the approach would become slow. A pruning strategy could stop at subsets whose partial sum already exceeds the target.
5. **No undo functionality.** There is no way to undo a move in Deck Casino. Since the game saves state to disk on demand, players can work around this by saving before risky moves, but a proper undo stack would be better.
6. **Move history is not persisted across sessions.** The move history is stored in the save file but is only displayed within the current session. Loading a saved game does not reload the history panel.

If the project were to continue, the highest-value improvements would be: (1) a proper hot-seat screen for multiplayer, (2) move undo, and (3) richer Baccarat animations.

10 Three Best and Three Weakest Parts

10.1 Three Strongest Parts

1. **Multi-group take validation with recursive backtracking.** The `_can_cover()` algorithm correctly handles the full generality of the Finnish Kasino rules, including the rare case where a single played card simultaneously captures three disjoint groups. This was the hardest algorithmic challenge in the project and is well-tested.
2. **Strict MVC separation with zero cross-layer leakage.** The model layer has no PyQt6 imports; the view layer has no game logic. This was enforced throughout development and makes the model fully testable from the command line. The `view.refresh(game)` refresh pattern is consistent across all three games.
3. **Hard AI sweep-avoidance placement.** When the hard AI cannot take any cards, it evaluates every candidate placement card against the resulting table state and selects the one that does not create a sweepable position for the opponent. This is non-trivial because it requires running the full `_can_cover` check for each candidate card and each possible hand value (2–16). The property test confirms this behaviour holds across full games.

10.2 Three Weakest Parts

1. **No GUI tests.** The view layer is entirely covered by manual testing. Any regression in the visual refresh logic can only be caught by running the application. Automated GUI testing would be a significant improvement.
2. **FileManager accesses private attributes.** The `restore()` method sets `game._players`, `game._table_cards`, etc. directly on the game object using `object.__new__(DeckCasinoGame)` to bypass `__init__`. This is a necessary workaround to restore a mid-round state without re-running `start_game()`, but it creates tight coupling between `FileManager` and `DeckCasinoGame` internals. A cleaner design would add a `DeckCasinoGame.from_state(data)` class method.
3. **AI does not look ahead more than one move.** The hard AI is greedy over one ply. A better AI would consider which cards it leaves on the table for the opponent and which cards it will likely draw next. Full lookahead is complex to implement correctly for a hidden-information game, so this was deprioritised in favour of correctness and speed.

11 Changes to the Original Plan

11.1 Planned Features Delivered

Every core feature from the original project plan was implemented, as summarised in Table 1.

Planned feature	Status
Finnish Deck Casino with multi-player support, sweep detection, special card values, and full scoring	Delivered
Blackjack: Hit, Stand, Double Down, Split	Delivered
Baccarat (Punto Banco): Punto/Banco/Tie bets, deterministic third-card rules	Delivered
CasinoTakeAlgorithm: enumerate all valid takes for a played card	Delivered
AIOpponent: Easy/Medium/Hard difficulty with 6-priority strategy	Delivered
Full mid-round save/load to JSON	Delivered
Bankroll persistence across Blackjack and Baccarat sessions	Delivered
MVC architecture: model has no PyQt6 imports; views have no game logic	Delivered
QPainter card rendering with no external image files	Delivered
Card deal and flip animations via QTimer	Delivered
Unit tests via unittest (plan required at least 10)	Delivered: 242 tests

Table 1: Planned features and their delivery status.

11.2 Implementation Differences from the Plan

CasinoTakeAlgorithm: recursive backtracking instead of plain `itertools`. The plan described a straightforward `itertools.combinations` subset-sum search. The final implementation adds a recursive backtracking layer (`_can_cover`) on top of that enumeration to validate multi-group takes: a single played card can simultaneously capture several disjoint table groups each summing to the card's hand value. This handles rule cases the simpler approach cannot.

No abstract Game base class. The plan defined an abstract `Game` superclass with `start_round`, `end_round`, and `get_valid_actions`. In practice, Deck Casino, Blackjack, and Baccarat have sufficiently different structures that a shared base class added little value. `GameManager` remains thin, and Blackjack and Baccarat are launched directly from `MainWindow`.

Player-roster-keyed save slots. The plan described a single `save_data.json`. The final implementation creates per-roster files (e.g. `save_Alice_Bob.json`) so different groups of players can maintain independent mid-round saves simultaneously.

11.3 Additional Features Beyond the Plan

- **Tutorial overlay:** a step-by-step guided walkthrough of Deck Casino rules, shown as an overlay directly on the table for new players.
- **Hint mode:** a toggle that displays each card's hand value as a badge, making the dual hand/table value system visible while learning.
- **Move history with improvement tips:** every Deck Casino action is recorded with an auto-generated tip (e.g. missed sweep warnings, special-card capture feedback), reviewable in the History panel.
- **Visual overlays:** `SweepFlashOverlay`, `RoundResultOverlay`, and `PointToastOverlay` provide in-table feedback drawn with `QPainter`, with no external assets.
- **Player badge widget:** a compact sidebar widget showing each player's name, score, and sweep count during Deck Casino, updating after every action.
- **Bankroll case-insensitive name matching:** when a name is entered that matches an existing bankroll entry only by capitalisation (e.g. "alice" vs. "Alice"), the player is prompted to choose whether to load that saved balance.

The time budget was approximately accurate for the model layer. The view layer took longer than planned because `QPainter` card rendering required more iteration than expected. Testing took less time than planned because the strict model/view separation made model tests straightforward to write.

12 Realised Order and Schedule

1. **Weeks 1–2 (early February):** Card, Deck, Hand, Player. Core model tests written in parallel.
2. **Week 3 (mid-February):** `CasinoTakeAlgorithm` with multi-group backtracking. `DeckCasinoGame` game loop (without scoring).

3. **Week 4 (late February):** Scoring logic, sweep detection, `end_round()`. Game-level integration tests.
4. **Week 5 (early March):** `BlackjackGame`, `BaccaratGame`, `BettingSystem`. Week-5 test suite.
5. **Week 6 (mid-March):** `AIOpponent` (all three difficulty levels). `MainWindow` and initial PyQt6 view scaffolding.
6. **Week 7 (late March):** `DeckCasinoView` with card selection and take/place controls. AI integration tests and property tests.
7. **Week 8 (early April):** `FileManager` save/load, bankroll persistence, move-history panel, tutorial overlay, card animations. Blackjack and Baccarat views.
8. **Weeks 9–10 (mid–late April):** Bug fixes, polish, documentation.

The implementation order followed the dependency graph: foundational model classes first, then game logic, then AI, then the GUI on top. This proved effective because the model was fully tested before the view was written, limiting debugging to one layer at a time.

13 Assessment of the Final Result

The project successfully delivers a functional three-game casino suite with a clean graphical interface. The primary game, Deck Casino, is fully implemented including the challenging multi-group take mechanic, three AI difficulty levels, save/load, and move history. Blackjack and Baccarat work correctly with proper payout mathematics and a shared persistent bankroll.

The strongest aspect of the implementation is the strict layering: the model is independently testable, and the 242-test suite gives high confidence that the game logic is correct. The hardest algorithmic challenge (multi-group take validation via recursive backtracking) is correctly implemented and thoroughly tested.

The main limitations are the lack of GUI tests, the direct private-attribute access in `FileManager.restore()`, and the one-ply depth of the AI. None of these affect correctness for typical gameplay, but they would need to be addressed before treating this as production-quality software.

The architecture is reasonably extensible: adding a fourth game would require only a new model class and a new view, with minimal changes to `MainWindow`. Adding lookahead to the AI would require only changes to `AIOpponent`, since the model's state is fully queryable.

14 References

1. Aalto University CS-A1123 course materials and project description. https://plus.cs.aalto.fi/y2/2026/project_topics/topics_pelit_104kasino_en/
2. Finnish Kasino card game rules document: `Project Overview/casino_card_game.md` (provided with the project topic).
3. PyQt6 documentation. <https://doc.qt.io/qtforpython-6/>. Used throughout the view layer; referenced in `view/main_window.py` and `view/card_widget.py`.
4. PyQt6 examples included in the course material (`PyQt6 Examples/` directory). Patterns for `QMainWindow`, `QStackedWidget`, and `QPainter` were adapted from these examples (see header comments in `view/main_window.py`).
5. Python standard library documentation: `itertools.combinations`, `json`, `random.shuffle`, `re.sub`. <https://docs.python.org/3/library/>
6. Blackjack basic strategy reference. The hard-total strategy table in `AIDealer._build_strategy()` was derived from the standard basic strategy as described in: Thorp, E. O. (1962). *Beat the Dealer*. Widely reproduced in summary form at <https://www.blackjackapprenticeship.com/blackjack-strategy-charts/>.
7. Punto Banco third-card rule table. The `BaccaratGame._banco_draws()` method implements the standard Punto Banco rule table as described at https://en.wikipedia.org/wiki/Baccarat#Third-card_rule.
8. Python `unittest` documentation. <https://docs.python.org/3/library/unittest.html>. Used for all unit and integration tests.